

Patch Arbitrary Code Execution #563

Closed

aaronclong opened on Dec 31, 2021 · edited by aaronclong

Edits ···

My company's internal deps auditing system is beginning to flag loguru because of this [potential exploit](#). I don't know if it has come to your attention.

I don't know exactly how to prevent it, but am willing to help out if you need support. I will study the issue more.

[418sec/huntr#1592](#)

2

1

aaronclong on Dec 31, 2021 · edited by aaronclong

Edits ··· Author

It looks like this [method](#) is the culprit: `_from_pickled_value`

I wonder if making it an instance method with mangling would solve the problem. A better solution would be to find another way to do this without **pickling**.

aaronclong on Dec 31, 2021

Author ···

As far as I can tell, this pickling is for multiprocessing that makes this so much more complex.

It was introduced with the 0.3.0 release in response to this issue: [#102](#)

Loguru to retrieve the module's name. From now, `record["name"]` will be equal to `None` in such case ([#62](#)).

- Fix logging methods not being serializable with `pickle` and hence raising exception while being passed to some `multiprocessing` functions ([#102](#)).

Delgan on Dec 31, 2021 · edited by Delgan

Edits ··· Owner

Hi [@aaronclong](#), thanks for reporting this issue.

The Huntr team contacted me some time ago about the vulnerability you're mentioning: [418sec#1](#)

I'm copy-pasting the comment I had left:

Hi.

Thanks for the security report. However, I'm not sure protecting `__reduce__()` in this case is really relevant. Sure, I understand that `pickle.loads()` is dangerous, but only when used with an untrusted source.

The `RecordException` is only meant to serialize Python errors. It will not be used again arbitrary data coming from network for example.

From what I see, the PoC is not different than calling `os.system()` in `__str__`:

```
import os

class MyClass:
    def __str__(self):
        os.system("xcalc")

logger.info(MyClass())
```



I am still quite confused by this report and I do not understand why Loguru is to blame.

The user receiving untrusted data should be responsible for sanitizing it before processing it. We can't expect this job to be done by the third-library, otherwise there might be infinite way to execute arbitrary code as demonstrated by the above example. This has little to do with the `pickle` module. Loguru isn't fetching and executing arbitrary code by itself, that's why it is difficult for me to recognize that there is a security problem in this library.

It is of course a pity that Loguru is reported as a dangerous library, and I wish it could be avoided. Perhaps you disagree with my analysis and can explain to me how my reasoning is wrong?

Of course, I'm in favor of improving Loguru safety and thanks for offering your help. However I want to understand the problem first and foremost to justify changes that may have multiple impacts (on features, performances and code complexity).

13

11

aaronclong on Dec 31, 2021

Author ...

I wonder if [dill](#) would be viable to work around this, since this library controls its own serialization.

aaronclong on Dec 31, 2021 · edited by aaronclong

Edits ▾ Author ...

@Delgan Since you are using this for exception handling, it is possible that an exception could trigger by a third-party library or whatever that would trigger it. Loguru would be just a catalyst in that workflow. Likewise, because it is in exceptions, it is extremely hard to sanitize that input.

My personal opinion that it is a logging library's responsibility to cover rogue execution. Using pickling is in general kind of dangerous.

aaronclong on Dec 31, 2021

Author ...

@Delgan if you don't have any plans on fixing this or remedying it, I understand. I would like to know though because I will need to remove it from my applications unfortunately.

Delgan on Dec 31, 2021

Owner ...

Since you are using this for exception handling, it is possible that an exception could trigger by a third-party library or whatever that would trigger it. Loguru would be just a catalyst in that workflow. Likewise, because it is in exceptions, it is extremely hard to sanitize that input.

Shouldn't the third-library with malicious `__reduce__` method be flagged instead?

My personal opinion that it is a logging library's responsibility to cover rogue execution. Using pickling is in general kind of dangerous.

How is it different from a third-library with malicious `__str__` method?

20

aaronclong on Dec 31, 2021

Author ...

@Delgan I agree with you that the third party should be flagged as well, but I don't agree that absolves your library in this case. It is on all sides, especially as zero trust principles are concerned.

3

aaronclong on Dec 31, 2021

Author ...

@Delgan likewise this code could in theory cause the same issue as the log4j exploit that happened this last month. It is quite similar to their arbitrary execution exploit.

Again, it's your library. No stress if this isn't the the path you want to go, I just need to know to make the appropriate changes on my end.

aaronclong on Dec 31, 2021

Author ...

@Delgan likewise this code could in theory cause the same issue as the log4j exploit that happened this last month. It is quite similar to their arbitrary execution exploit.

Again, it's your library. No stress if this isn't the the path you want to go, I just need to know to make the appropriate changes on my end.

Delgan on Dec 31, 2021 · edited by Delgan

Edits ▾ Owner ...

As I said, I'm all in favor of improving Loguru's security, but I don't know how to do it in this case without compromising the features.

The exception needs to be somehow serialized. The exception comes from user's code. I can't restrain the allowed attributes during serialization because it would unfairly limit usability for users with custom and trusted exceptions.

I'm genuinely trying to understand. Do you think Loguru should be responsible for analyzing `__str__` method of every argument before displaying it?

Here is another example:

```
import os
from multiprocessing import Pool

class Malicious:
    def __reduce__(self):
        os.system("firefox")
        return (Malicious, ())

def f(x):
    return 0

if __name__ == "__main__":
    with Pool(processes=4) as pool:
        pool.map(f, [Malicious()])
```

Should `multiprocessing` be flagged because it can execute arbitrary code?

aaronclong on Dec 31, 2021 · edited by aaronclong

Edits ▾ Author ⋮

@Delgan I think `__str__` is different that `__reduce__`. `__reduce__` being automatically triggered by the interpreter causes problems. These are just long standing design flaws within python that unfortunately we have to patch over.

I don't know necessarily how to solve this problem, maybe dill or another format would make this less problematic. It could also be a custom internal format that is simple and easy to serialize/deserialize. We could even look into how the built in logger handles this scenario. I am certain we can find other resources for recommendation.

As for multiprocessing, it is a library with some flaws in it's design. However, I understand the need to support it. In general, there are people that choose to use alternatives to the built in multiprocessor like [pathos](#). I am neither for nor against multiprocessing, but just want to acknowledge those issues.

Delgan on Dec 31, 2021

Owner ⋮

Sorry for maybe sounding too defensung , by the way. It's just that I truly don't understand how `pickle` would be different that any other method in this very specific case. I feel every library is flagged as unsafe just for using `pickle`. ☺

For example, `__str__` can also be "automatically triggered by the interpreter":

```
import os

class Malicious:
    def __str__(self):
```

```
os.system("firefox")
return "M()"
```

```
m = Malicious()
string = f"M: {m}"
```

Anyway, initially you were only trying to solve a problem reported by a third party, so I don't want to bother you further. Thanks for answering my questions. :)

I had already looked for safe alternatives to `pickle` but I didn't find anything that suited me. The `dill` library offers more features than `pickle` but [suffers from the same vulnerability issues](#). The standard `logging` library also [uses pickle similarly](#) to share the `record` to another process (which is also why Loguru needs to serialize the exception). Again, sorry to insist, but Loguru is just using `pickle` to serialize an user's class, so it's not different if the malicious code lived in the `__reduce__` of another user's class.

I'll try to get in touch with Huntr team for discussing this issue further.

♥ 5

Delgan on Dec 31, 2021 · edited by Delgan

Edits ▾

Owner

...

Also I don't think it's fair to compare this issue with the Log4Shell exploit. To clarify, there is no way for Loguru to execute arbitrary code from user string input alone.

▮ 26

aaronclong on Dec 31, 2021 · edited by aaronclong

Edits ▾

Author

...

@Delgan it is being flagged for pickle and how it exposes pickle. To be clear, pickle isn't a safe method of serialization. It is a problematic legacy format.

Unfortunately, you can convert a pickle response to string and back again. Not saying it is the most likely scenario, but it is possible. Yes, it is quite similar to the log4j issue because it allowed for Java Objects to be loaded dynamically by the logger from another source. The string expansion was only one half of the equation. **For more context, read [here](#).**

28 remaining items

Load more

wmealing on Jan 31, 2022

...

It's disconcerting to me that such an issue can make it's way into a CVE, then into a github advisory.

There is a method to 'dispute' a CVE, please let me know if you want this followed up as I am very familiar with this process.

♥ 3



v1a0 added a commit that references this issue on Jan 31, 2022

security issue fixed, [CVE-2022-0329](#) ...

f30d43b

orf on Jan 31, 2022 · edited by orf

Edits ▾ ...

There is a method to 'dispute' a CVE, please let me know if you want this followed up as I am very familiar with this process.

I think that would be up to [@Delgan](#), no? If not then I'd be interested in knowing more about the process

wmealing on Jan 31, 2022 · edited by wmealing

Edits ▾ ...

Mitre has it documented here:

https://cve.mitre.org/cve/list_rules_and_guidance/correcting_counting_issues.html#dispute

Specifically the part:

Not everyone shares the same definition of a vulnerability. One person's vulnerability is another person's security hardening opportunity, and another person's intended functionality. How does CVE deal with these differing opinions?

When an authoritative source disputes the validity of the vulnerability, "*** DISPUTED ***" is added to the beginning of the description, and a short NOTE is added to the end explaining why the vulnerability is disputed. Ideally, the disputing party provides a link that can be added to the CVE as a reference, and a quote that can be used as the explanation in the NOTE. However, neither are required.

You can lodge a dispute here: <https://cveform.mitre.org/>

Choose "Request a update to e CVE Entry"

Type of update: "Other"

My advice is to make a github issue or wiki page separate to this outlining the specifics of why is not an issue, and lock it. (People will almost definitely have 'input' on this, and you want the message to be your definitive voice, nobody else)

State that you'd like it to be disputed (you can't reject it, only the original CVE submitter can do that, sadly) and provide a link to your description on why this is not a flaw. Also, you can provide them with a small blurb that explains why it is not a flaw for their page.

This 'small blurb' will become visible on the CVE pages and tooling (and of course, vulnerability scanners) showing the correct state of the flaw and the current *DISPUTED* status. This can take up to two weeks, but Mitre is pretty good at it, they -may- refer you to github as the CNA and put you in contact with the github CNA (https://www.cve.org/PartnerInformation/ListofPartners/partner/GitHub_M)

I do not know the original reporter of the flaw but contacting github on the top address may be able to put you in contact with the original reporter who -may- request REJECTION of the CVE.. but I've never had a reporter ever reject a CVE in multiple attempts over 10 years of working with flaws.

If you have any more questions wmealing @ < #ff0000 > hat.com . Thanks.



renovate mentioned this in 5 pull requests [on Jan 31, 2022](#)

chore(deps): update dependency loguru to v0.6.0 [security] - autoclosed [lpsm-dev/dummy-port-scanner#15](#)

Update dependency loguru to v0.6.0 [SECURITY] - autoclosed [dixneuf19/fuzzy-octo-disco#30](#)

Update dependency loguru to v0.6.0 [SECURITY] - autoclosed [dixneuf19/dank-face-bot#21](#)

Update dependency loguru to v0.6.0 [SECURITY] - autoclosed [hackersandslackers/aiohttp-aiofiles-tutorial#13](#)

Update dependency loguru to v0.6.0 [SECURITY] - autoclosed [dalekurt/python-users-service#27](#)

carnil on Feb 4, 2022

The CVE was rejected apparently now.

8

2

wmealing on Feb 4, 2022 via email · edited by wmealing

Edits ...

Posted to wrong github issue. Removing.



dwenjii mentioned this [on Feb 17, 2022](#)

[loguru 0.5.3 vulnerability CVE-2022-0329 mpenning/ciscoconfparse#223](#)



Delgan mentioned this [on Sep 1, 2023](#)

[Fix errors due to non-unpicklable Exception when "enqueue=True" #963](#)



sonnyp added 2 commits that reference this issue [on Jun 18, 2025](#)

build(deps): bump the controller group in /controller with 5 updates ...

b1f4ef5

build(deps): bump the segmenter group in /segmenter with 4 updates (#1 ...)

e8bf0b1

Sign up for free to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Assignees

No one assigned

Labels

enhancement

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

 Code with agent mode

▼

No branches or pull requests

Participants



+7